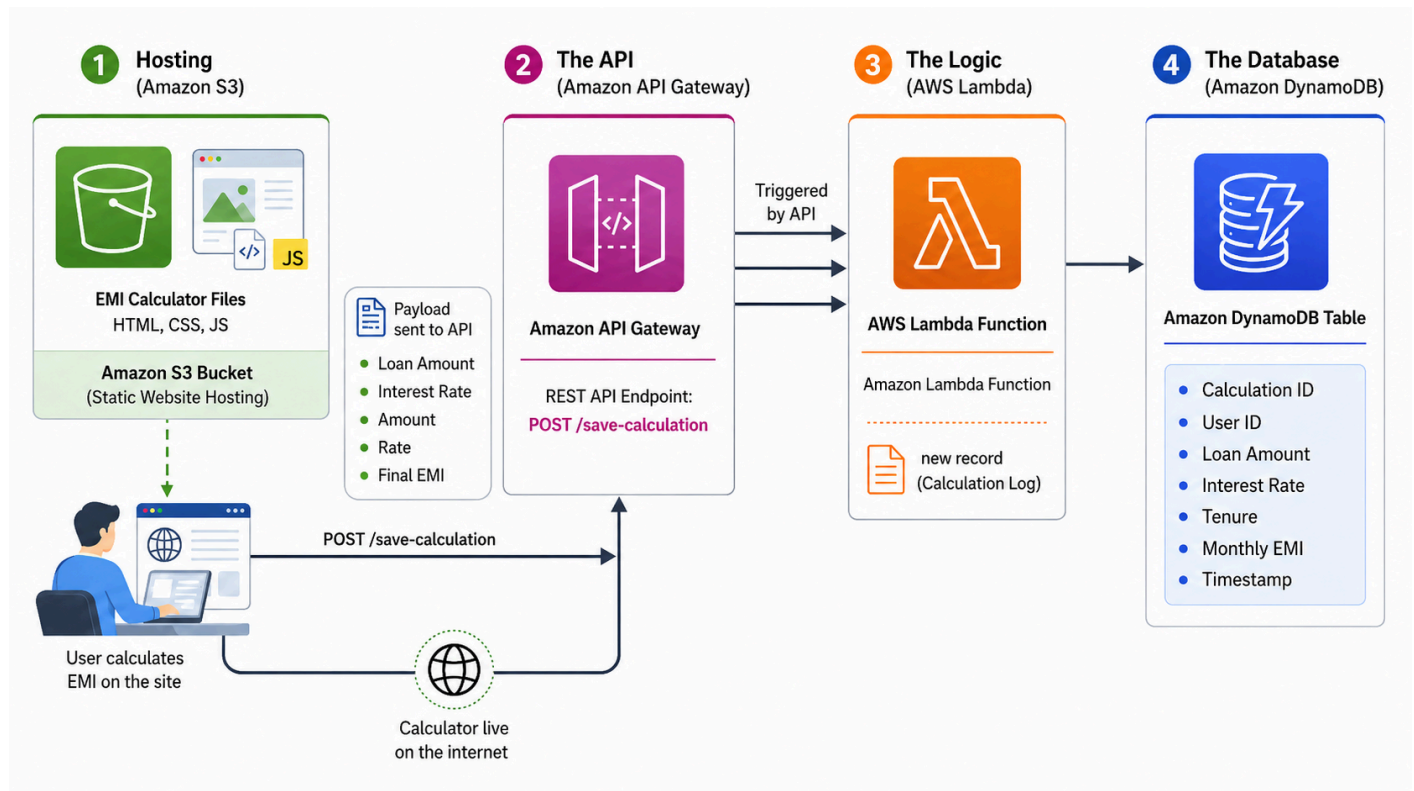


Cloud Hosted EMI Calculator: Serverless Architecture Migration

Objective: To migrate a static, client-side EMI calculator into a fully functional, serverless web application so as to persist user data or track calculation history.

Solution: Implemented an AWS serverless backend (API Gateway, Lambda, DynamoDB) to capture, process, and permanently store calculation records while maintaining a highly responsive, globally distributed frontend via Amazon S3.



Phase 1: The Database (Amazon DynamoDB)

The DynamoDB table stores the history of calculations on the EMI Calculator Web Application.

1) Table details:

- Table name: EMI_History
- Partition key: calculationId (Type: String). This will be a unique ID for every calculation.

2) Table settings: On-Demand capacity mode Under "Read/write capacity settings". This ensures payment as per usage, exactly \$0.00 while the table sits idle.

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

EMI_History

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

calculationid String

1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key name String

1 to 255 characters and case sensitive.

Table Creation in DynamoDB: Table Name & Partition Key

Table class [Info](#)

Select table class to optimize your table's cost based on your workload requirements and data access patterns.

Choose table class

☒ **DynamoDB Standard**
The default general-purpose table class. Recommended for the vast majority of tables that store frequently accessed data, with throughput (reads and writes) as the dominant table cost.

☐ **DynamoDB Standard-IA**
Recommended for tables that store data that is infrequently accessed, with storage as the dominant table cost.

Capacity calculator [Info](#)

Read/write capacity settings [Info](#)

Capacity mode

☒ **On-demand**
Simplify billing by paying for the actual reads and writes your application performs.

☐ **Provisioned**
Manage and optimize your costs by allocating read/write capacity in advance.

Maximum table throughput - optional

Specify maximum on-demand read or write (or both) throughput for tables and secondary indexes to help keep your throughput usage and costs bounded. By default, maximum table throughput does not apply and on-demand throughput is only limited by the default DynamoDB table quotas.

Table Creation in DynamoDB: Table Class & Read/Write Capacity Settings

Phase 2: The Compute Logic (AWS Lambda & IAM)

The Lambda function receives data from the frontend (EMI Calculator Web Application) and saves it to the DynamoDB table (**EMI_History**). Also, an IAM Role is created for the Lambda function so that it can put data into the DynamoDB table.

1) IAM Role details:

- Role name: EMICalculatorLambdaRole
- Entity type: AWS service
- Use case: Lambda

- Permissions policies: **AWSLambdaBasicExecutionRole** (so it can log to CloudWatch)
- Inline policy: An inline policy **PutItemDynamoDB** is created with dynamodb:PutItem on the **ARN** of the **EMI_History** table. It is attached to **EMICalculatorLambdaRole** which allows the Lambda function to put items in the DynamoDB table.

The screenshot shows the 'Select trusted entity' step in the AWS IAM console. The left sidebar indicates the current step is 'Step 1: Select trusted entity'. The main area is titled 'Select trusted entity' and contains two sections: 'Trusted entity type' and 'Use case'.

Trusted entity type

- ☒ **AWS service**: Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**: Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**: Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**: Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**: Create a custom trust policy to enable others to perform actions in this account.

Use case

Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case

Lambda

Choose a use case for the specified service.

Use case

- ☒ **Lambda**: Allows Lambda functions to call AWS services on your behalf.

Role Creation in IAM: Entity Type & Use Case

The screenshot shows the 'Add permissions' step in the AWS IAM console. The left sidebar indicates the current step is 'Step 2: Add permissions'. The main area is titled 'Add permissions' and contains a section for 'Permissions policies (1/1214)'.

Permissions policies (1/1214)

Choose one or more policies to attach to your new role.

Filter by Type

Search: lambdabasic (X) All types 24 matches

	Policy name	Type	Description
☐	AWSLambdaBasicDurableExecutionRolePolicy	AWS managed	Provides write permissions to CloudWa...
☒	AWSLambdaBasicExecutionRole	AWS managed	Provides write permissions to CloudWa...
☐	AWSLambdaBasicExecutionRole-01e42d4d-867...	Customer managed	-
☐	AWSLambdaBasicExecutionRole-0301187d-0eca...	Customer managed	-
☐	AWSLambdaBasicExecutionRole-0b8ac340-5c85...	Customer managed	-
☐	AWSLambdaBasicExecutionRole-2062299b-dc3...	Customer managed	-
☐	AWSLambdaBasicExecutionRole-2d740969-a9b...	Customer managed	-
☐	AWSLambdaBasicExecutionRole-30fec08c-9920...	Customer managed	-

Role Creation in IAM: Permissions Policies

aws Search [Alt+S] Global awswithantara (426271381760) antadmin

IAM > Roles > Create role

Step 1 Select trusted entity
Step 2 Add permissions
Step 3 **Name, review, and create**

Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.

Maximum 64 characters. Use alphanumeric and '+,=,@,-' characters.

Description
Add a short explanation for this role.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=, @-/\[\]\!#\$%^&*(){}~`"

Role Creation in IAM: Role Name & Description

aws Search [Alt+S] Ask Amazon Q Global awswithantara (426271381760) antadmin

IAM > Policies > Create policy

DynamoDB
Allow 1 Action

Specify what actions can be performed on specific resources in **DynamoDB**.

Actions allowed
Specify actions from the service to be allowed.

Write
☒ PutItem [Info](#)

Resources
Specify resource ARNs for these actions.
☐ All
☒ Specific

table	Info
arn:aws:dynamodb:ap-south-1:426271381760:table/EMI_History	

☐ Any in this account
[Add ARNs to restrict access.](#)

Request conditions - optional
Actions on resources are allowed or denied only when these conditions are met.

Policy Creation in IAM: Actions Allowed on DynamoDB with specific ARN

aws Search [Alt+S] Ask Amazon Q Global awswithantara (426271381760) antadmin

IAM > Policies > Create policy

Step 1 Specify permissions
Step 2 **Review and create**

Review and create [Info](#)

Review the permissions, specify details, and tags.

Policy details

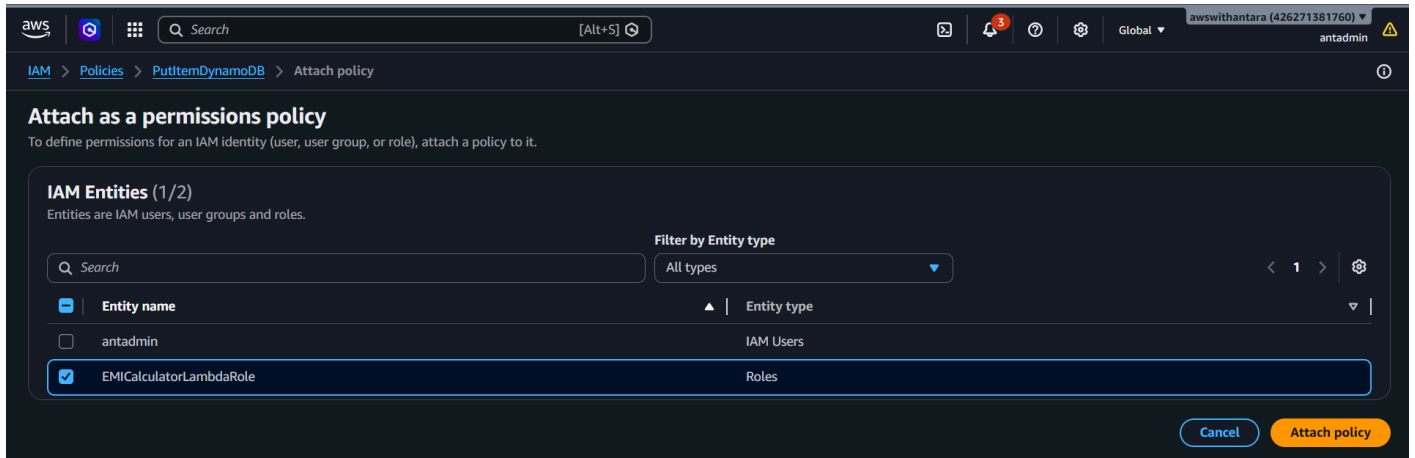
Policy name
Enter a meaningful name to identify this policy.

Maximum 128 characters. Use alphanumeric and '+,=,@,-' characters.

Description - optional
Add a short explanation for this policy.

Maximum 1,000 characters. Use alphanumeric and '+,=,@,-' characters.

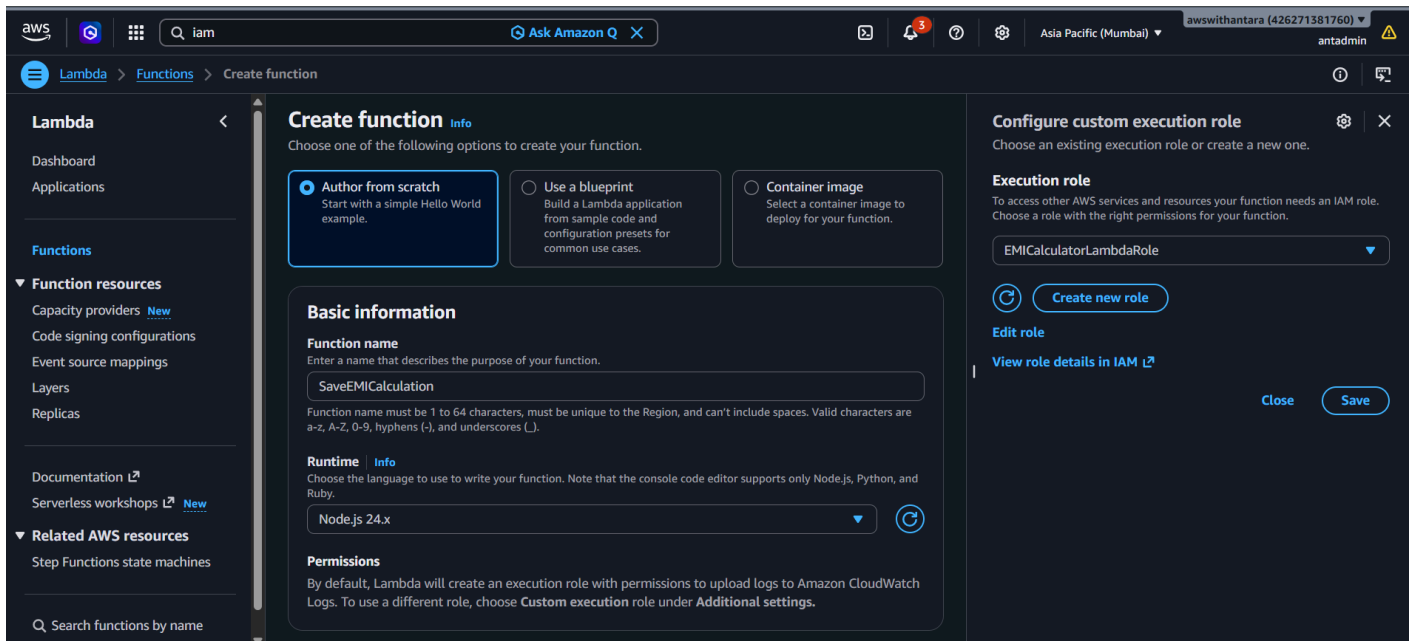
Policy Creation in IAM: Policy Name and Description



Edited IAM Policy: Attach the IAM Policy to the IAM Role

2) Lambda Function Details:

- Name: SaveEMICalculation
- Runtime: Nodejs 24.x
- Execution Role: Custom created execution role(IAM Role) **EMICalculatorLambdaRole**



Lambda Function Creation: Name, Runtime & Custom Execution Role

Lambda Code:

JavaScript

```
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import { DynamoDBDocumentClient, PutCommand } from "@aws-sdk/lib-dynamodb";
import { randomUUID } from "crypto";

const client = new DynamoDBClient({});
const dynamo = DynamoDBDocumentClient.from(client);
const tableName = "EMI_History";

export const handler = async (event) => {
  // 1. Log the raw incoming request (Viewable in AWS CloudWatch)
  console.log("Incoming request body:", event.body);

  let body;
  try {
    // Parse the JSON string sent from your frontend fetch()
    body = JSON.parse(event.body);
  } catch (parseError) {
    return {
      statusCode: 400,
      headers: { "Access-Control-Allow-Origin": "*" },
      body: JSON.stringify({ error: "Invalid JSON payload format" })
    };
  }

  // 2. Map the incoming frontend payload to DynamoDB attributes
  const params = {
    TableName: tableName,
    Item: {
      calculationId: randomUUID(),
      timestamp: new Date().toISOString(),
      principalAmount: body.principal,
      interestRate: body.rate,
      tenureMonths: body.tenure,
      calculatedEMI: body.emi,
      totalInterestOutgo: body.totalInterest,
      totalOverallPayment: body.totalPayment
    }
  }
```

```

};

// 3. Execute the database save
try {
    await dynamo.send(new PutCommand(params));

    return {
        statusCode: 200,
        // CORS headers are strictly required here for your S3 hosted
        site to accept the response
        headers: {
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Methods": "OPTIONS,POST",
            "Access-Control-Allow-Headers": "Content-Type"
        },
        body: JSON.stringify({ message: "Calculation history saved to
AWS!" })
    };
} catch (error) {
    console.error("DynamoDB Write Error:", error);

    return {
        statusCode: 500,
        headers: { "Access-Control-Allow-Origin": "*" },
        body: JSON.stringify({ error: "Failed to connect to the
database" })
    };
}
};

```

Phase 3: The Bridge(API Gateway)

The frontend (EMI Calculator Web Application) cannot communicate with lambda directly. It needs an API endpoint.

1) REST API Details:

- Name: EMICalculatorAPI
- API endpoint type: Regional

Create REST API [Info](#)

API details

☒ **New API**
Create a new REST API.

☐ **Clone existing API**
Create a copy of an API in this AWS account.

☐ **Import API**
Import an API from an OpenAPI definition.

☐ **Example API**
Learn about API Gateway with an example API.

API name
EMICalculatorAPI

Description - optional
API for my Frontend Web Application

API endpoint type
Regional

Security policy - new [Info](#)
Transport Layer Security (TLS) protects data in transit between a client and server. The security policy also determines the cipher suite options that clients can use with your API.
Choose a security policy

API Gateway: REST API Creation

2) Resource Details:

- Resource path: /
- Resource name: calculate

Create resource

Resource details

☐ **Proxy resource** [Info](#)
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

Resource path
/

Resource name
calculate

☐ **CORS (Cross Origin Resource Sharing)** [Info](#)
Create an OPTIONS method that allows all origins, all methods, and several common headers.

[Cancel](#) [Create resource](#)

REST API Resource Creation: Resource Path & Name

3) Method Details:

- Method type: POST
- Integration type: Lambda Function
- Lambda proxy integration: Must be enabled, it passes the entire HTTP request to the Lambda code.

Create method

Method details

Method type: POST

Integration type:

- ☒ **Lambda function**
Integrate your API with a Lambda function.
- ☐ HTTP
Integrate with an existing HTTP endpoint.
- ☐ Mock
Generate a response based on API Gateway mappings and transformations.
- ☐ AWS service
Integrate with an AWS Service.
- ☐ VPC link
Integrate with a resource that isn't accessible over the public internet.

☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Response transfer mode: **Buffered**
Wait to receive the complete response before beginning transmission.

Method Creation for REST API Resource: Method & Integration type, Lambda Proxy

Response transfer mode | Info

- ☒ **Buffered**
Wait to receive the complete response before beginning transmission.
- ☐ Stream
Send portions of the response without waiting for the complete response.

Lambda function
Provide the Lambda function name or alias. You can also provide an ARN from another account.

ap-south-1 | arn:aws:lambda:ap-south-1:426271381760:function:SaveEMI

Grant API Gateway permission to invoke your Lambda function
When you save these changes, API Gateway updates your Lambda function's resource-based policy to allow this API to invoke it.

Integration timeout | Info
By default, you can enter an integration timeout of 50 - 29,000 milliseconds. You can use Service Quotas to raise the integration timeout to greater than 29,000 ms. When the response transfer mode is streaming, the valid range is 50 - 900,000 milliseconds and cannot be increased further.

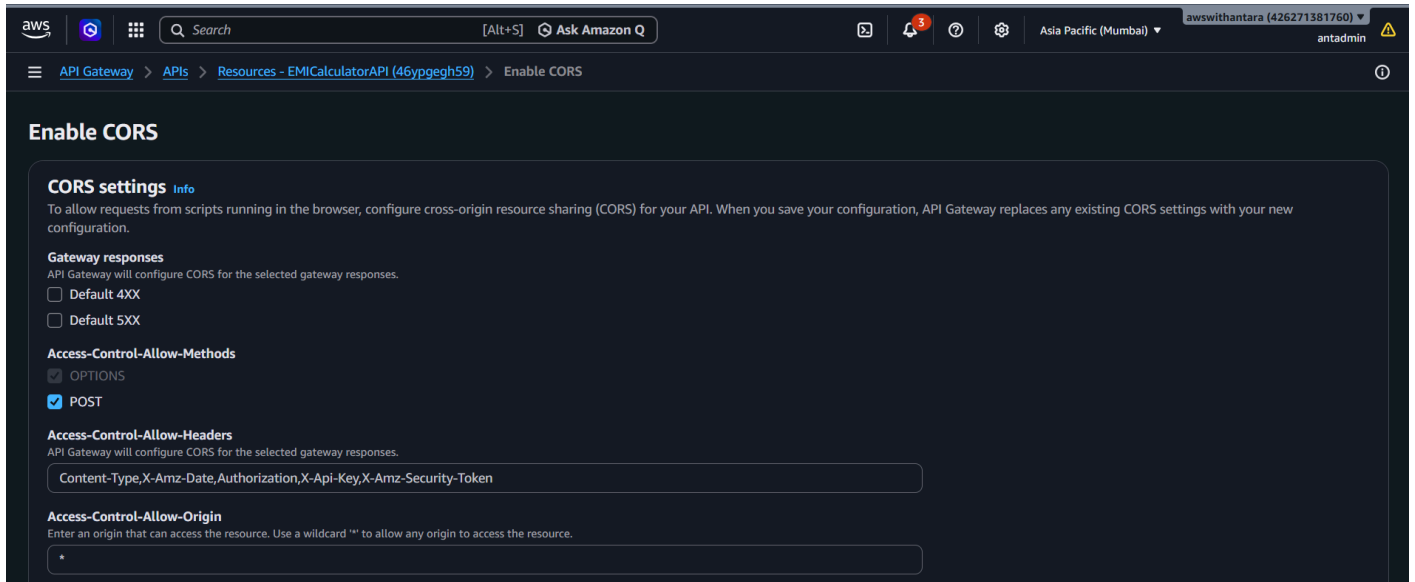
29000

Method Creation for REST API Resource: Lambda Function Integration

4) CORS (Cross Origin Resource Sharing) Details:

- Access Control Allow Methods: OPTIONS & POST
- Access Control Allow Headers: Must include Content-Type so as to match frontend fetch request headers
- Access Control Allow Origin: * (any origin)

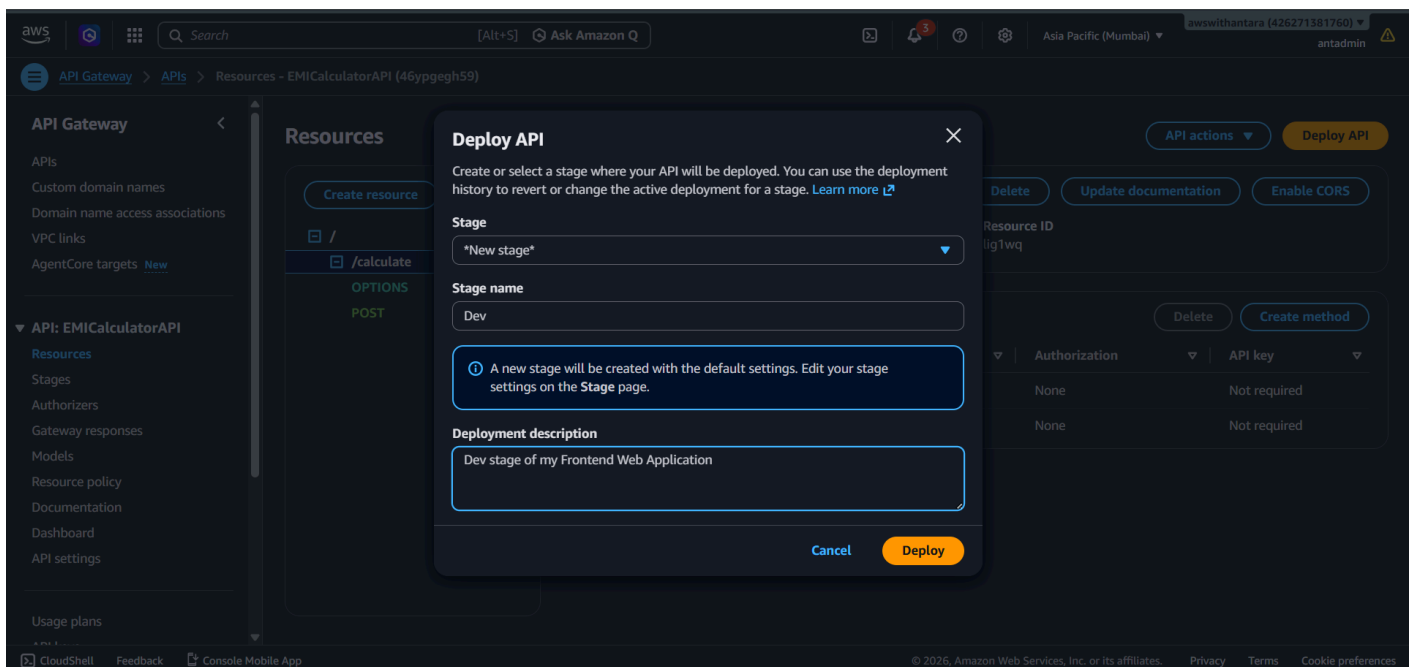
CORS is enabled and configured for the **/calculate** resource because such permissions are tied to specific endpoints. When the frontend JavaScript makes a fetch request to the API URL, the web browser first sends an invisible "preflight" request to that exact path which must explicitly grant permissions.



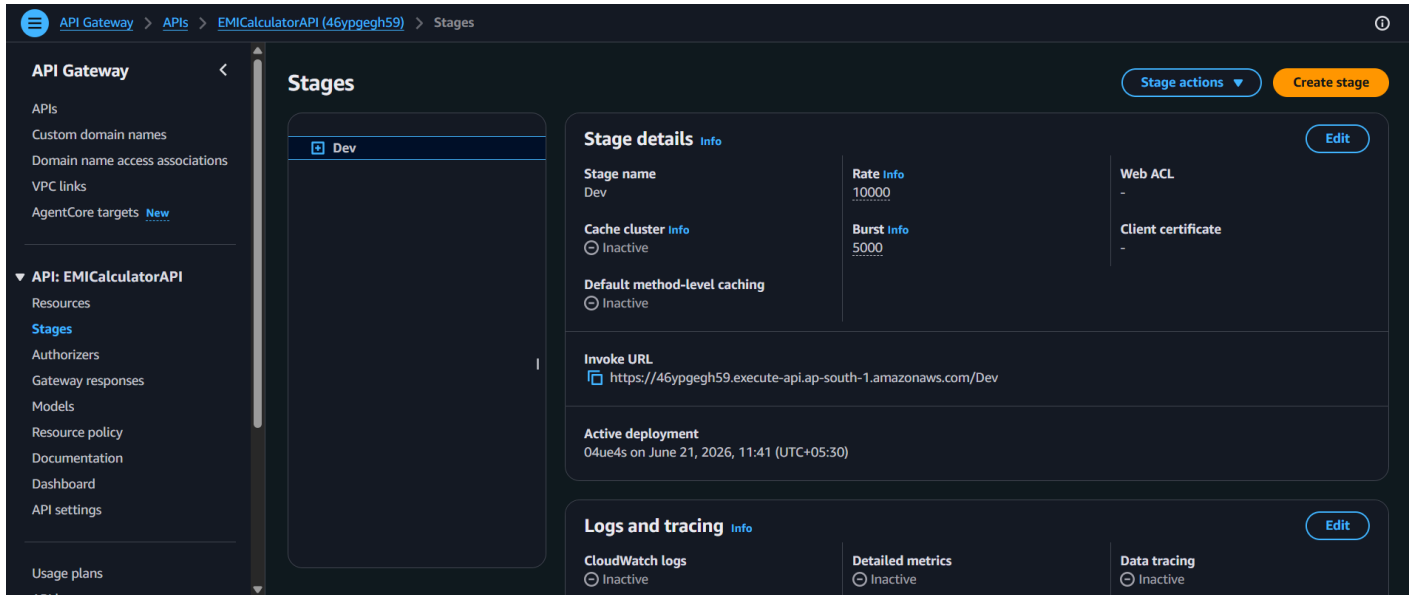
CORS Configuration for REST API Resource: Access Control Allow Settings

5) Deploy API:

- Stage: *New Stage*
- Stage Name: Dev
- Description: Dev stage of my frontend web application



REST API Deploy: Stage Creation



REST API Dev Stage: Stage Details

Phase 4: Frontend Integration & Request Optimization

The Frontend Web Application is connected to the API Gateway (**EMICalculatorAPI**) without causing rate-limiting or excessive AWS resource consumption from live interactions.

1) Implementation Details:

- Formatted a JSON payload inside **script.js** of the frontend application extracting the live state of Principal, Rate, Tenure, EMI, Total Interest, and Total Payment.
- Implemented an asynchronous **fetch()** POST request targeting the live API Gateway **/calculate** resource.

2) Performance Optimization:

- The frontend UI includes active sliders that fire rapid recalculations, hence a Debounce function is integrated (**setTimeout**)
- This strategy prevents API Gateway spam, mitigating DDoS-like traffic patterns, and strictly minimizing resource consumption thereby reducing accumulated costs.

Frontend JavaScript Integration (script.js):

JavaScript

```
// — AWS Cloud Sync (Phase 4 Integration - Debounced) —————
// Clear the previous timer so it doesn't spam AWS while sliding
clearTimeout(cloudSyncTimeout);

// Set a new timer to wait 2 seconds after the user stops sliding
cloudSyncTimeout = setTimeout(() => {
  const payload = {
    principal: principal,
    rate: rate,
    tenure: tenure,
    emi: emi,
    totalInterest: totalInterest,
    totalPayment: totalPayment
  };

  fetch("https://96u69oekg4.execute-api.ap-south-1.amazonaws.com/dev/calculate", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(payload)
  })
  .then(response => response.json())
  .then(data => console.log("AWS Sync Success:", data))
  .catch(error => console.error("AWS Sync Error:", error));
}, 2000);
```

Phase 5: Amazon S3 Global Hosting

An S3 Bucket is provisioned comprising the Frontend (EMI Calculator Web Application) assets as objects, and configured explicitly for Static Website Hosting.

1) S3 Bucket Details:

- Name: amzn-cloud-emi-calculator-2026-antara
- Object Ownership: ACLs disabled (recommended)
- Block all public access: Must be disabled to allow public access (enabled by default)

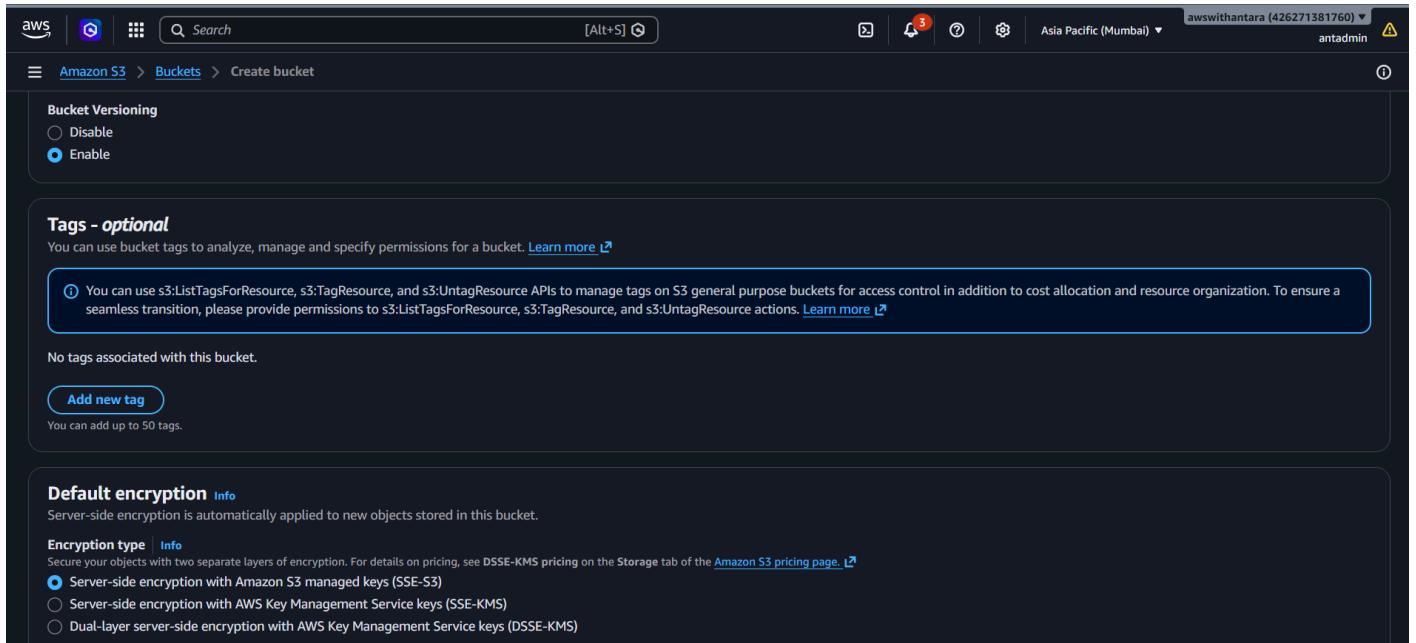
- **Bucket Versioning:** Must be enabled in order to prevent accidental deletions of data from the bucket. **Delete Markers** can be used with this feature to recover data in case of such deletions.

The screenshot shows the 'Create bucket' page in the AWS Management Console. The breadcrumb navigation is 'Amazon S3 > Buckets > Create bucket'. The page title is 'Create bucket' with an 'info' icon. A subtitle states 'Buckets are containers for data stored in S3.' The 'General configuration' section is active. Under 'AWS Region', 'Asia Pacific (Mumbai) ap-south-1' is selected. Under 'Bucket type', 'General purpose' is selected with a radio button, and a description explains it's the original S3 bucket type. The 'Directory' option is also visible. Under 'Bucket namespace', 'Global namespace' is selected with a radio button, and a description explains it's the default. The 'Account Regional namespace (recommended)' option is also visible. The 'Bucket name' field contains 'amzn-cloud-emi-calculator-2026-antara'. A note below the field states that bucket names must be 3 to 63 characters and unique. At the bottom, there is a 'Copy settings from existing bucket - optional' section and a 'Choose bucket' button.

S3 Bucket Creation: Bucket type & name

The screenshot shows the 'Create bucket' page in the AWS Management Console, specifically the 'Object Ownership' and 'Block Public Access settings' sections. The breadcrumb navigation is 'Amazon S3 > Buckets > Create bucket'. The page title is 'Object Ownership' with an 'info' icon. A subtitle states 'Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.' Under 'Object Ownership', 'ACLs disabled (recommended)' is selected with a radio button, and a description explains that all objects are owned by this account. The 'ACLs enabled' option is also visible. Under 'Block Public Access settings for this bucket', a subtitle explains that public access is granted through ACLs, bucket policies, access point policies, or all. A note states that turning on 'Block all public access' is the same as turning on all four settings below. The 'Block all public access' checkbox is checked. Below it, four individual settings are listed, each with a checkbox: 'Block public access to buckets and objects granted through new access control lists (ACLs)', 'Block public access to buckets and objects granted through any access control lists (ACLs)', 'Block public access to buckets and objects granted through new public bucket or access point policies', and 'Block public and cross-account access to buckets and objects through any public bucket or access point policies'. All four individual settings are also checked.

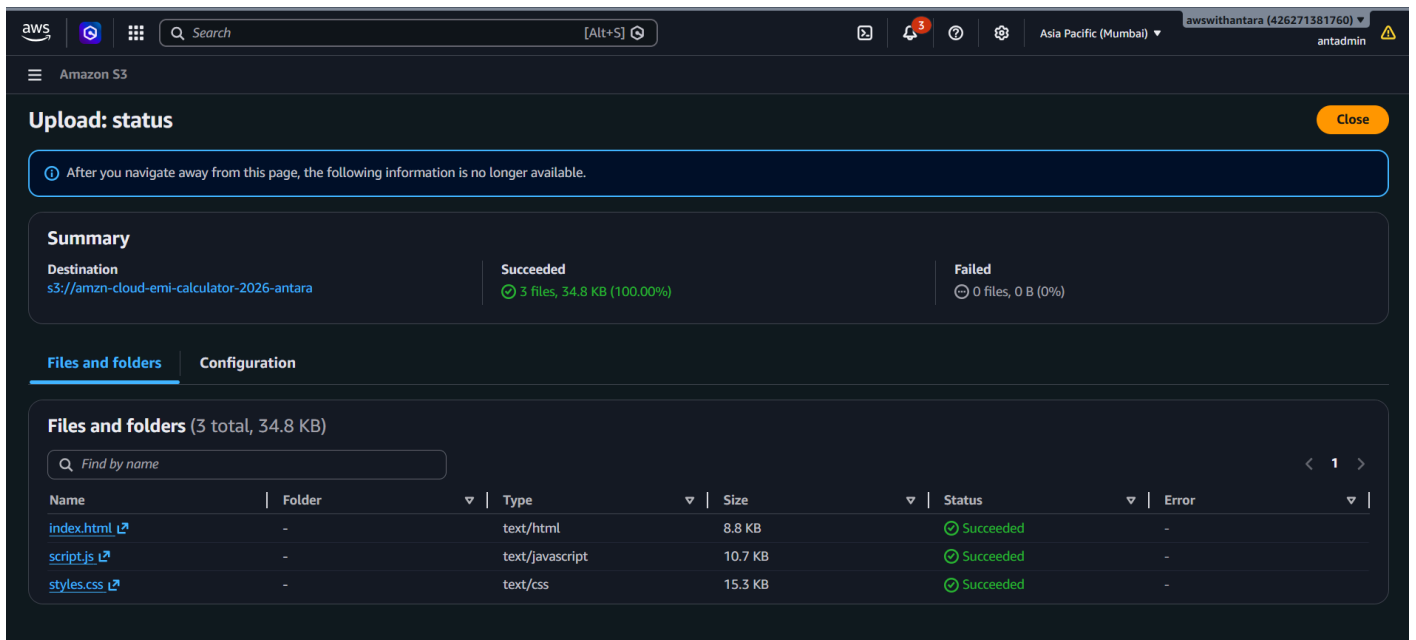
S3 Bucket Creation: Object ownership & Block all public access



S3 Bucket Creation: Bucket versioning & encryption

2) Objects Upload:

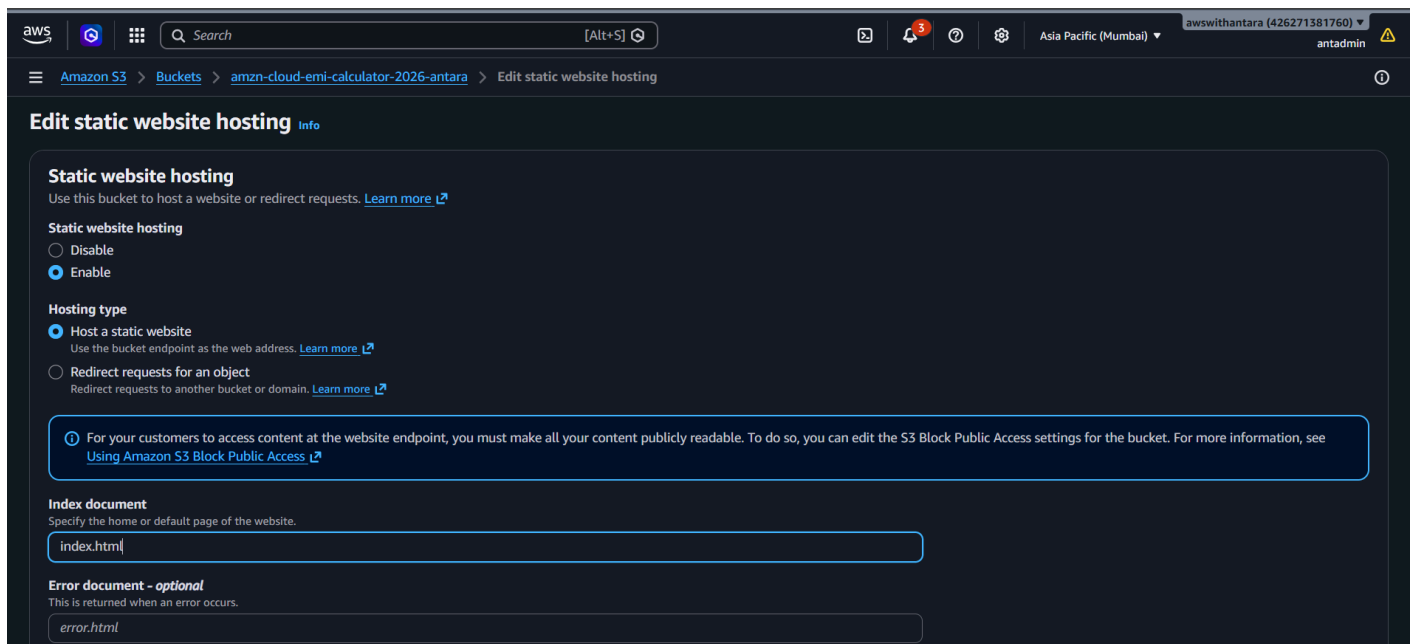
- The Frontend assets (**index.html**, **styles.css** & **scripts.js**) are uploaded in the newly created bucket **amzn-cloud-emi-calculator-2026-antara**



S3 Bucket: Objects upload status

3) Static Website Hosting:

- Hosting type: Host a static website
- Index document: index.html



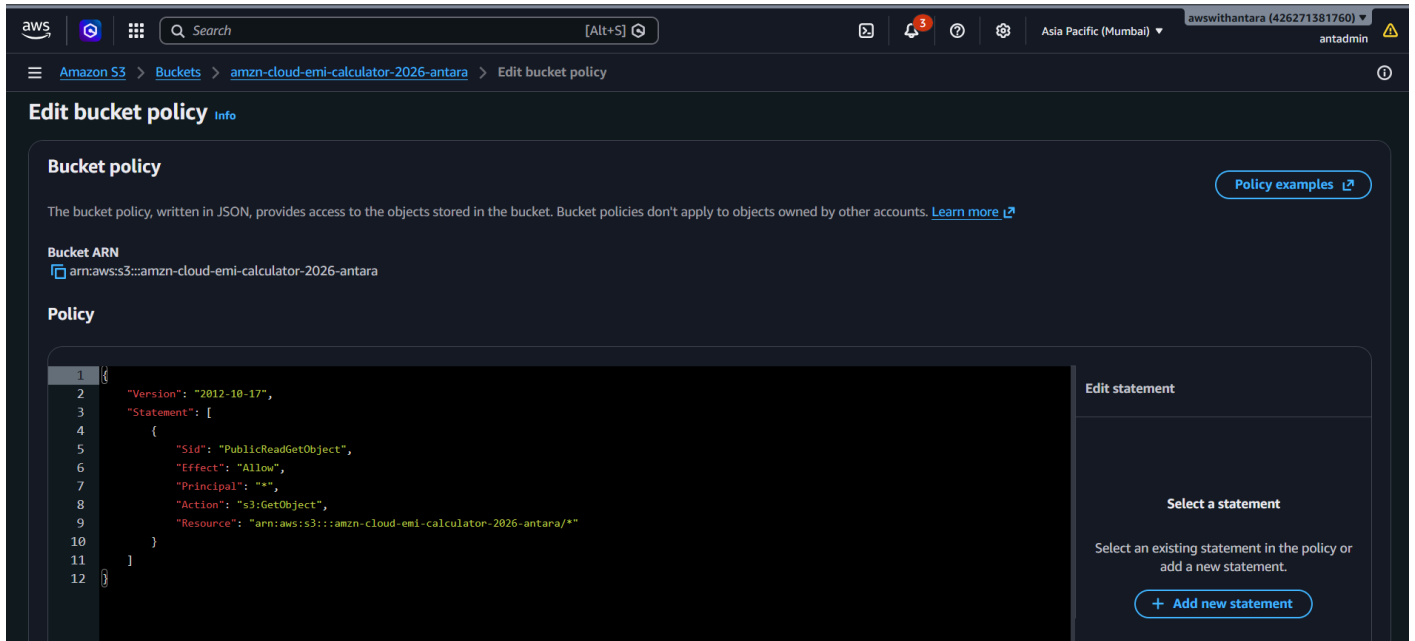
S3 Bucket: Static website hosting

4) Bucket Policy:

- In addition to disabling **Block all public access**, a bucket policy has to be defined in order to explicitly allow frontend access to any user on the internet.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource":
"arn:aws:s3:::amzn-cloud-emi-calculator-2026-antara/*"
    }
  ]
}
```



S3 Bucket: Bucket policy

Inference: This serverless architecture demonstrates a complete evolution from a static, client-side interface into a dynamic, cloud-native web application.

- By decoupling the architecture, this project achieves high availability and ultra-low latency on the frontend via Amazon S3 global edge hosting, while maintaining a secure, scalable, and stateful backend via API Gateway, AWS Lambda, and DynamoDB.
- Furthermore, the implementation of client-side request debouncing paired with On-Demand database capacity highlights a strong focus on cloud economics and resource optimization.